

CS132S Data Structures & Algorithms

Test #1

Test 1 In Class and Test 1 Take Home are open-book (course textbook) and open-notes (your own notes from this class) and open-laptop, but closed-neighbor. You may NOT communicate with anyone about the Test before the Test 1 Take Home submission deadline, other than the Instructor. Do not forget the Academic Dishonesty Policy.

Relax, take a deep breath, and pray ☺.

Your grade will be based upon both the correctness and quality of your solutions. Don't get stuck on one portion of the test, but look all of it over first. If you cannot completely solve something, do your best, and submit whatever you are able to, so that I will have more opportunity to award any applicable partial credit.

DO NOT LEAVE THE ROOM until I verify to you that your submission for Test 1 In Class has been received successfully.

For the IN CLASS PORTION of this Test, implement solutions to ANY TWO (your choice) of the following problems. If you choose to implement solutions to more than two of these problems, you will receive normal credit for the two you do the best, plus a little bit of extra credit for each additional one of sufficient quality.

For the TAKE HOME PORTION of this Test, implement solutions to ANY FOUR of the following problems.

(continued on next page)

1. Deliverable: Test1_1.py

Sample Input/Output:

```
>>>
```

```
RESTART: /Users/jeickemeyer/TMU/Courses/CS132S/_key/Test1_1.py
This program finds (the more efficiently, the more credit)
whether or not a random list has any repeated elements.
```

```
Enter random minrange: 1
Enter random maxrange: 100
Enter length of list: 10
```

```
[36, 20, 12, 65, 98, 82, 68, 86, 14, 39] has no repeated elements.
```

```
>>>
```

```
RESTART: /Users/jeickemeyer/TMU/Courses/CS132S/_key/Test1_1.py
This program finds (the more efficiently, the more credit)
whether or not a random list has any repeated elements.
```

```
Enter random minrange: 1
Enter random maxrange: 10
Enter length of list: 5
```

```
[8, 9, 1, 9, 8] has repeated elements.
```

```
>>>
```

2. Deliverable: Test1_2.py

Sample Input/Output:

```
>>>
  RESTART: /Users/jeickemeyer/TMU/Courses/CS132S/_key/Test1_2.py
Enter Postfix Expression: A B C * + D +
Converted to Prefix is: + + A * B C D
>>>
  RESTART: /Users/jeickemeyer/TMU/Courses/CS132S/_key/Test1_2.py
Enter Postfix Expression: A B + C D + *
Converted to Prefix is: * + A B + C D
>>>
  RESTART: /Users/jeickemeyer/TMU/Courses/CS132S/_key/Test1_2.py
Enter Postfix Expression: A B * C D * +
Converted to Prefix is: + * A B * C D
>>>
  RESTART: /Users/jeickemeyer/TMU/Courses/CS132S/_key/Test1_2.py
Enter Postfix Expression: A B + C + D +
Converted to Prefix is: + + + A B C D
>>>
```

3. Deliverable: Test1_3.py

Sample Input/Output:

>>>

RESTART: /Users/jeickemeyer/TMU/Courses/CS132S/_key/Test1_3.py
Enter word list: behold we count them happy which endure

Radix Pass 0: behold we count them happy which endure
Radix Pass 1: we count them happy which behold endure
Radix Pass 2: we them which behold endure count happy
Radix Pass 3: we which them count behold happy endure
Radix Pass 4: we endure them behold which happy count
Radix Pass 5: happy we behold them which endure count
Radix Pass 6: behold count endure happy them we which

>>>

RESTART: /Users/jeickemeyer/TMU/Courses/CS132S/_key/Test1_3.py
Enter word list: diligently seeketh good procureth favour

Radix Pass 0: diligently seeketh good procureth favour
Radix Pass 1: seeketh good procureth favour diligently
Radix Pass 2: seeketh good favour procureth diligently
Radix Pass 3: seeketh good favour procureth diligently
Radix Pass 4: good favour procureth seeketh diligently
Radix Pass 5: good diligently favour procureth seeketh
Radix Pass 6: good seeketh diligently favour procureth
Radix Pass 7: procureth good diligently seeketh favour
Radix Pass 8: seeketh diligently procureth good favour
Radix Pass 9: favour seeketh diligently good procureth
Radix Pass 10: diligently favour good procureth seeketh

>>>

RESTART: /Users/jeickemeyer/TMU/Courses/CS132S/_key/Test1_3.py
Enter word list: labourer is worthy of his reward

Radix Pass 0: labourer is worthy of his reward
Radix Pass 1: is worthy of his reward labourer
Radix Pass 2: is worthy of his reward labourer
Radix Pass 3: is of his reward labourer worthy
Radix Pass 4: is of his reward labourer worthy
Radix Pass 5: is of his reward labourer worthy
Radix Pass 6: is of labourer worthy his reward
Radix Pass 7: labourer reward of his worthy is
Radix Pass 8: his is labourer of reward worthy

>>>

RESTART: /Users/jeickemeyer/TMU/Courses/CS132S/_key/Test1_3.py
Enter word list: choose you this day whom ye will serve

Radix Pass 0: choose you this day whom ye will serve
Radix Pass 1: you this day whom ye will serve choose
Radix Pass 2: you this day whom ye will serve choose
Radix Pass 3: you day ye will whom choose this serve
Radix Pass 4: ye this will whom choose serve you day
Radix Pass 5: day ye serve this whom choose will you
Radix Pass 6: choose day serve this whom will ye you

>>>

4. Deliverable: Test1_4.py

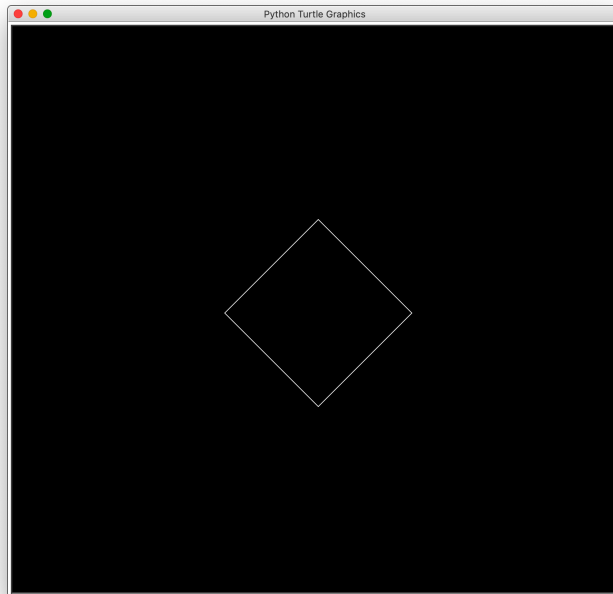
Sample Input/Output:

```
>>>
```

```
RESTART: /Users/jeickemeyer/TMU/Courses/CS132S/_key/Test1_4.py
```

```
Enter recursion levels: 0
```

```
Click turtle screen to exit...
```

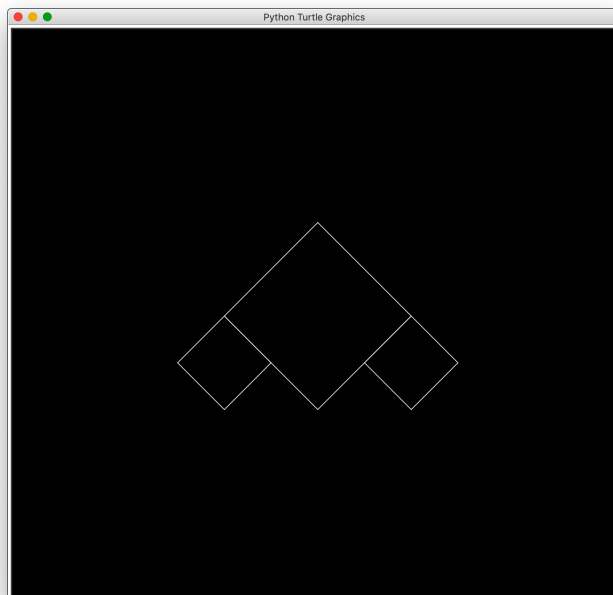


```
>>>
```

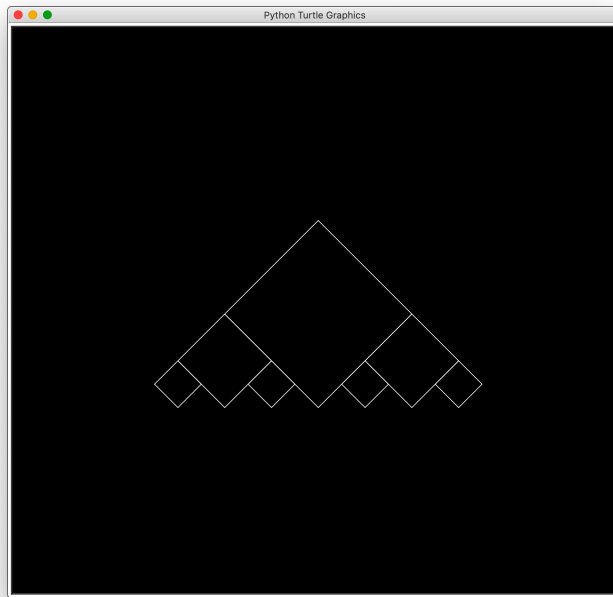
```
RESTART: /Users/jeickemeyer/TMU/Courses/CS132S/_key/Test1_4.py
```

```
Enter recursion levels: 1
```

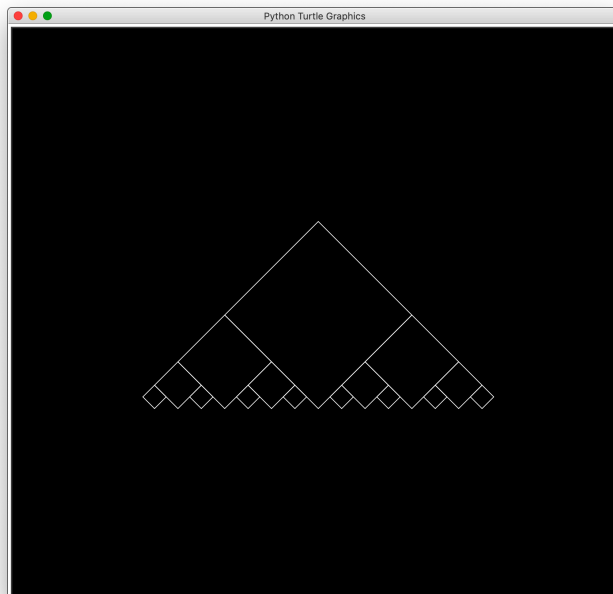
```
Click turtle screen to exit...
```



```
>>>
  RESTART: /Users/jeickemeyer/TMU/Courses/CS132S/_key/Test1_4.py
Enter recursion levels: 2
Click turtle screen to exit...
```



```
>>>
  RESTART: /Users/jeickemeyer/TMU/Courses/CS132S/_key/Test1_4.py
Enter recursion levels: 3
Click turtle screen to exit...
```



```
>>>
```

5. Deliverable: Test1_5.py

Sample Input/Output:

```
>>>
RESTART: /Users/jeickemeyer/TMU/Courses/CS132S/_key/Test1_5.py
This program computes, using dynamic programming,
the Mystery Function M(n,k):
M(n,k) = k*M(n-1,k-1) + M(n-1,k)
M(n,0) = 1 for all n >= 0
M(i,j) = 0 for all j > i

Enter n: 4
Enter k: 2
The value of M(4,2) is 12
>>>
RESTART: /Users/jeickemeyer/TMU/Courses/CS132S/_key/Test1_5.py
This program computes, using dynamic programming,
the Mystery Function M(n,k):
M(n,k) = k*M(n-1,k-1) + M(n-1,k)
M(n,0) = 1 for all n >= 0
M(i,j) = 0 for all j > i

Enter n: 1000
Enter k: 500
The value of M(1000,500) is
3297886364098853712202425207011626170699648569798169281898590320909424797
1781752051776286414428286064251161003889658118202680594069109645716381914
6404867449323195951016063077387633734854267300606090269861265477271286488
5767612974840440214546353633972363886005212952635701939963538010202890374
8262849320184896141812161981977276499303352299663586434433390642957548013
2415225196248339593869027449698810760918514647597480713398046665485357617
3839395952080663634453441149497402104807214024765380034893206439625039781
5943867930883023403478254538199466448753235746806520129555368911829594558
1602629007720957929180752212770840487049163879535858780678097564405860520
5612517083370745142043254311005906164376194932253533513852823895263272543
0316247573500509629637791057644011092351061147845564061689200193082413487
9251166941326675998088653284459628349959991040037803289845575791602454929
7784488730729642813507147565842082259370467068397779183834492055819551311
9257793638378369609375572753298679225801330828486642168050234850213758240
9888931707193522417988009028652278091249663021542543131557667939046775828
8574982435057080928851405383111561186074237123394470525957961694090942450
7145817061031002275700840584632126549228321933586439990537045779153934071
4797690375415057559712073706121243182307015204282485816648002227404800000
000000000000000000000000000000000000000000000000000000000000000000000000
000000000000000000000000000000000000000000000000000000000000000000000000
>>>
```

~~~ END OF CS132S TEST #1 ~~~